

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Дальневосточный федеральный университет»
(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Департамент математики и компьютерного моделирования

Отчет
к лабораторной работе №7 по дисциплине «Суперкомпьютеры и параллельное программирование»

По теме: «Типы потоков. Управление потоками»

Вариант 20

Выполнил: студент группы Б9122-01.03.02мкт

_____ Чирич И.С.
подпись *Ф.И.О.*

Проверил: Доцент

_____ Селезнев Т.Э.
подпись *Ф.И.О*

г. Владивосток
2025

Содержание

Постановка задачи **3**

Индивидуальное задание **4**
Написание программного кода 4
Тестирование 7

Вопросы для защиты работы **9**

Выводы **10**

Постановка задачи

Цель данной лабораторной работы: освоить принципы создания и управления потоками.

Задачи лабораторной работы:

- Научиться создавать фоновые и приоритетные потоки
- Научиться работать с пулом потоков
- Научиться решать практические задачи с использованием пула потоков

Индивидуальное задание

Написание программного кода

Создадим шаблон консольного проекта при помощи утилиты `dotnet`:

```
dotnet new console -n thread_types
```

В корне проекта создастся файл Project.cs, в котором и будет написан код лабораторной работы.

Создаем класс Program с методом Main:

```
class Program
{
        static void Main(string[] args)
        {

        }
}
```

В соответствии с индивидуальным заданием определим тип `PairVec`, который содержит информацию о паре векторов случайной длины. Спецификация типа для выполнения задания:

- Идентификатор для определения обрабатываемого потоком элемента
- Численная константа `MULTIPLIER`, которая определяет множитель для случайно сгенерированного вещественного числа
- Длина векторов
- Три вектора: два сгенерированных и один - результат их сложения

Описание типа в виде программного кода в листинге ниже:

```
public class PairVec
{
        public int id;
        public double MULTIPLIER = 100_000;
        public int n;
        public List<double> v1;
        public List<double> v2;
        public List<double> result;
        public PairVec(int id, int n)
        {
                this.id = id;
                this.n = n;
        }
}
```

Создадим коллекцию элементов типа `PairVec`, которая содержит метод `Add` для сложения векторов, и метод `ThreadCalc` для запуска параллельных вычислений:

```
public class Array
{
        List<PairVec> list;
        public Array()
        {
                this.list = new List<PairVec>();
        }
        public void Add(PairVec pv)
        {
                list.Add(pv);
                Console.WriteLine("Добавлен элемент по индексу {0}", list.Count - 1);
                ThreadPool.QueueUserWorkItem(ThreadCalc, pv);
        }
        private static void ThreadCalc(object o)
        {
                PairVec pv = (PairVec)o;
                Console.WriteLine("Поток: {1}. В пул потоков попал вектор id = {0}...",
                     pv.id,
                Thread.CurrentThread.ManagedThreadId);
                Console.WriteLine("Поток: {1}. Генерируем пару векторов id = {0}...",
                    pv.id,
                Thread.CurrentThread.ManagedThreadId);
                Random random = new Random();
                pv.v1 = new List<double>(pv.n);
                pv.v2 = new List<double>(pv.n);
                pv.result = new List<double>(pv.n);
                for (int i = 0; i<pv.n; i++)
                {
                        pv.v1.Add(random.NextDouble()* pv.MULTIPLIER);
                        pv.v2.Add(random.NextDouble()* pv.MULTIPLIER);
                }
                for (int i = 0; i<pv.n; i++)
                {
                        pv.result.Add(pv.v1[i] + pv.v2[i]);
                }
                Thread.Sleep(1000);
                Console.WriteLine("Поток: {1}. Сумма векторов элемента вычисленна id =
                    {0}", pv.id,
                Thread.CurrentThread.ManagedThreadId);
        }
}
```

Отметим, что класс `Array` есть обертка над встроенной реализацией списка - `System.Collections.Generic.List`.

Опишем функцию `Main`. Она создает инстанс кастомной коллекции и добавляет объекты пользовательского типа:

```
static void Main(string[] args)
{
        const int ARRSIZE = 5;
```

```
        const int VECSIZE = 100_000;
        Array array = new Array();
        for (int i = 0; i < ARRSIZE; i++)
        {
                array.Add(new PairVec(id: i, n: VECSIZE));
        }
        Console.ReadKey();
}
```

Итоговый код в листинге ниже:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace thread_types
{
        public class PairVec
        {
                public int id;
                public double MULTIPLIER = 100_000;
                public int n;
                public List<double> v1;
                public List<double> v2;
                public List<double> result;
                public PairVec(int id, int n)
                {
                        this.id = id;
                        this.n = n;
                }
        }

        public class Array
        {
                List<PairVec> list;
                public Array()
                {
                        this.list = new List<PairVec>();
                }
                public void Add(PairVec pv)
                {
                        list.Add(pv);
                        Console.WriteLine("Добавлен элемент по индексу {0}", list.Count
                             - 1);
                        ThreadPool.QueueUserWorkItem(ThreadCalc, pv);
                }
                private static void ThreadCalc(object o)
                {
                        PairVec pv = (PairVec)o;
                        Console.WriteLine("Поток: {1}. В пул потоков попал вектор id =
```

```
                        {0}...", pv.id,
42                      Thread.CurrentThread.ManagedThreadId);
43                      Console.WriteLine("Поток: {1}. Генерируем пару векторов id =
                            {0}...", pv.id,
44                      Thread.CurrentThread.ManagedThreadId);
45                      Random random = new Random();
46                      pv.v1 = new List<double>(pv.n);
47                      pv.v2 = new List<double>(pv.n);
48                      pv.result = new List<double>(pv.n);
49                      for (int i = 0; i<pv.n; i++)
50                      {
51                              pv.v1.Add(random.NextDouble()* pv.MULTIPLIER);
52                              pv.v2.Add(random.NextDouble()* pv.MULTIPLIER);
53                      }
54                      for (int i = 0; i<pv.n; i++)
55                      {
56                              pv.result.Add(pv.v1[i] + pv.v2[i]);
57                      }
58                      Thread.Sleep(1000);
59                      Console.WriteLine("Поток: {1}. Сумма векторов элемента
                            вычисленна id = {0}", pv.id,
60                      Thread.CurrentThread.ManagedThreadId);
61              }
62      }
63
64      internal class Program
65      {
66              static void Main(string[] args)
67              {
68                      const int ARRSIZE = 5;
69                      const int VECSIZE = 100_000;
70                      Array array = new Array();
71                      for (int i = 0; i < ARRSIZE; i++)
72                      {
73                              array.Add(new PairVec(id: i, n: VECSIZE));
74                      }
75                      Console.ReadKey();
76              }
77      }
78  }
```

Тестирование

Запуск программы осуществим командой, которая скомпилирует проект и запустит его в среде выполнения:

```
1  dotnet build; dotnet run
```

Вывод программы:

```
Добавлен элемент по индексу 0
Добавлен элемент по индексу 1
Добавлен элемент по индексу 2
Добавлен элемент по индексу 3
Добавлен элемент по индексу 4
Поток: 4. В пул потоков попал вектор id = 0...
Поток: 4. Генерируем пару векторов id = 0...
Поток: 8. В пул потоков попал вектор id = 4...
Поток: 8. Генерируем пару векторов id = 4...
Поток: 3. В пул потоков попал вектор id = 2...
Поток: 7. В пул потоков попал вектор id = 3...
Поток: 5. В пул потоков попал вектор id = 1...
Поток: 5. Генерируем пару векторов id = 1...
Поток: 7. Генерируем пару векторов id = 3...
Поток: 3. Генерируем пару векторов id = 2...
Поток: 5. Сумма векторов элемента вычисленна id = 1
Поток: 8. Сумма векторов элемента вычисленна id = 4
Поток: 3. Сумма векторов элемента вычисленна id = 2
Поток: 7. Сумма векторов элемента вычисленна id = 3
Поток: 4. Сумма векторов элемента вычисленна id = 0
```

Программа выполняется корректно. Осуществляется вывод ключевых событий потоков с указанием как номера потока, так и идентификатора обрабатываемого элемента.

Вопросы для защиты работы

1. **Какие типы потоков вы знаете? Чем они отличаются?**

   По приоритету разделяют:

   - Фоновые: указываются через свойство объекта `IsBackground = true`
   - Основные: указываются через свойство объекта `IsBackground = false`

   Отличие в том, что программа завершится только после того, как доработают основные потоки. Фоновые же завершатся автоматически при отсутствии основных потоков.

2. **Для чего применяются приоритеты потоков? Как задать приоритет потока?**

   Приоритет - величина, указывающая операционной системе относительный размер процессорного времени, необходимый потоку. Чем больше величина - тем больше времени получит процесс.
   Приоритет задается через свойство `Priority` объекта типа `Thread`. Оно принимает одно из значений перечисления `ThreadPriority`.

3. **Что такое пул потоков? Какой метод запускает поток в пуле?**

   Пул потоков - структура, управлемая средой выполнения `CLR`, которая дает возможность абстрагироваться от управления множеством потоков. Она позволяет эффективно раздавать задачи потокам, не тратя ресурсы на пересоздание нужного числа потоков.
   Для запуска кода в пуле потоков используется метод
   `ThreadPool.QueueUserWorkItem`

Выводы

В данной лабораторной работе изучен один из эффективных способов работы с потоками - через объект пула потоков `ThreadPool`.